Person A: Cluster and Infrastructure

Complete 2-day work plan (main PC owner)

Project: Global Earthquake Analytics and Seismic Pattern Detection (HDFS 3-node cluster, Spark on YARN, Hive, MongoDB)

Item	Detail
Your role	Build and run the 3-node cluster on the main PC, load the dataset into HDFS, set up Hive and MongoDB, help B and C run their jobs, and collect all cluster evidence the course requires. You also keep the GitHub repository healthy.
Already done	WSL2 limited to 9 GB and 4 CPUs (Docker shows 8.7 GiB); Docker Desktop engine running; `hello-world` works; images `ubuntu:22.04` and `mongo:7` pulled.
You depend on	**Person B** for `dataset/quakes.csv` (full file) by Day 1 afternoon.
Who depends on you	**Everyone.** B and C cannot run on the cluster until your Day 1 steps A4 to A7 work.
Your repo folders	`config/`, `scripts/`, `src/ingestion/`, `screenshots/`, `docs/cluster-setup.md`, `docs/failover-test.md`
Minimum commits	5 required by the course. Plan for 10 or more (section 9).
Hours	About 9 hours per day (09:00 to 18:00), two days.

What you will deliver

File or evidence	Day	Purpose
GitHub repo + folder skeleton	1	Repo created at the start, all 3 members added
`config/Dockerfile`, `config/docker-compose.yml`	1	The 3-node cluster plus MongoDB
`config/hadoop|spark|hive/*`	1	Node configuration (course item 1)
`scripts/*.sh`	1	Start, init and status scripts
`src/ingestion/load_to_hdfs.sh`	1	Upload the dataset to HDFS with 16 MB blocks
Cluster status screenshots	1	Cluster status (course item 2)
Block distribution screenshot	1	Storage distribution (course item 3)
Spark on YARN screenshots	1, 2	Execution of the analytics task (course item 4)
Hive and MongoDB working	1, 2	Tools demanded by the project topic
`docs/failover-test.md` + screenshots	2	Bonus evidence: the cluster survives a node loss
`docs/demo-script.md`	2	What you say and show in the final demo

Contents

1 Ground rules

2 What the cluster looks like

3 Day 1 tasks (A1 to A9)

4 Day 2 tasks (A10 to A15)

5 Screenshot checklist (the 5 things the course asks to see)

6 Serving B and C (hand-off and remote access)

7 Daily operation cheat sheet

8 Commit plan

9 Demo narration

10 Viva questions you should be able to answer

11 Troubleshooting

12 Final repo check

1. Ground rules

- Commit after every finished task, from your own GitHub account, using `git` on **Windows** (not inside a container). The repo folder is mounted live into the master container at `/opt/project`.
- Do not edit B's or C's files. If their script fails on the cluster, send them the error text and the output of `docker logs master`.
- Never run two heavy Spark jobs at the same time. The whole cluster has about 8.7 GB.
- Never use `docker compose down -v` unless you want to **delete HDFS data, the Hive metastore and MongoDB data**. Plain `docker compose stop` and `start` keep everything.
- Keep the Docker Desktop window closed and the PC plugged in (power plan: High performance; sleep: never).
- Close Chrome tabs, IDEs and other heavy apps while the cluster runs.

Note: Version contract for the team: **Hadoop 3.3.6, Spark 3.5.1, Hive 3.1.3, Java 11, Python 3.10** (the container uses Ubuntu 22.04). B and C install `pyspark==3.5.1` on their laptops to match.

2. What the cluster looks like

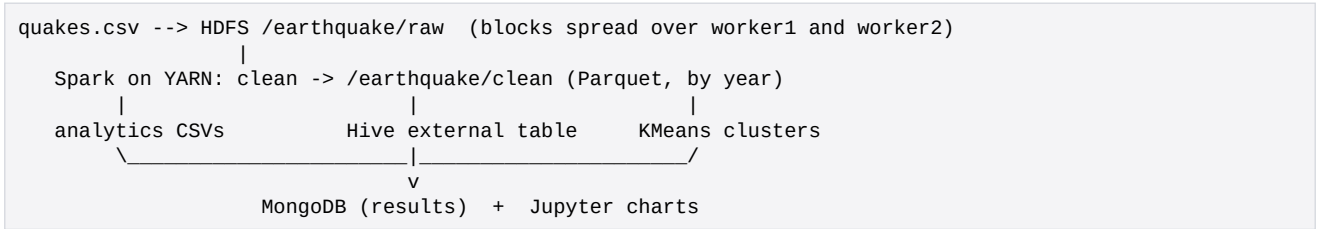
Four Docker containers on one network called `hadoop`. The same image (`eq-hadoop`) runs `master`, `worker1` and `worker2`; the `ROLE` environment variable decides which daemons start.

Container	Daemons	Memory cap	Ports on Windows
`master`	HDFS NameNode, YARN ResourceManager, Spark driver (client mode), HiveServer2	2.7 GB	9870 (HDFS UI), 8088 (YARN UI), 4040 (Spark UI), 10000 (Hive)
`worker1`	HDFS DataNode, YARN NodeManager (runs Spark executors)	2.7 GB	9864, 8042
`worker2`	HDFS DataNode, YARN NodeManager (runs Spark executors)	2.7 GB	9865, 8043
`mongo`	MongoDB 7	0.5 GB	27017

Key settings (know these for the viva)

Setting	Value	Why
`dfs.replication`	2	Only 2 DataNodes exist; every block is stored on both
`dfs.blocksize`	16 MB	A 100 MB file becomes 7 blocks, so the spread over workers is visible
`yarn.nodemanager.resource.memory-mb`	2304	Fits one 1 GB executor (1.5 GB with overhead) plus the small application master
`maximum-am-resource-percent`	0.6	Default 0.1 would leave Spark jobs stuck in ACCEPTED on a tiny cluster
`spark.yarn.jars`	hdfs:///spark-jars/*.jar	Jars uploaded once, so each job starts in seconds instead of uploading 300 MB
Spark mode	YARN, client	Executors run on the workers; the driver on the master
Hive metastore	Embedded Derby	Simple and enough for one HiveServer2

Data flow of the whole project:



3. Day 1 tasks

Goal for the end of Day 1: the cluster is running, the full dataset is in HDFS spread over both workers, a Spark job has run on YARN, Hive and MongoDB answer, and B and C have access.

A1. Create the repo and folder skeleton (09:00 to 09:45)

1 On GitHub create a new **private** repository named `BigDataProject`.

2 Settings, Collaborators: add Person B and Person C (their GitHub usernames). Ask them to accept.

3 Clone it to `C:\Users\Lenovo\BigDataProject` (Git for Windows required: `git --version`).

4 Create the folder skeleton from the course brief and push.

```
cd C:\Users\Lenovo
git clone https://github.com/<owner>/BigDataProject.git
cd BigDataProject
git config user.name "Your Name"
git config user.email "your-github-email"
mkdir dataset src\ingestion src\processing src\analytics scripts config output screenshots docs pipeline
```

Git does not track empty folders, so add a `.gitkeep` file in each one, and create `.gitignore`:

```
for %d in (dataset src\ingestion src\processing src\analytics scripts config output screenshots docs
pipeline) do type nul > %d\.gitkeep
```

```
# .gitignore  (create this file in the repo root)
dataset/quakes.csv
dataset/*.zip
.venv/
__pycache__/
*.pyc
.ipynb_checkpoints/
config/downloads/
```

```
git add -A
git commit -m "A: create repo skeleton and gitignore"
git push
```

Note: Run the `for` line above in **Command Prompt**. In Git Bash use `touch dataset/.gitkeep` and so on. Tell B and C to `git clone` now so they can start.

A2. Add the cluster files and read them (09:45 to 11:00)

Unzip `cluster-files.zip` into the repo root. You get `config/`, `scripts/`, `docs/cluster-setup.md`, `.dockerignore` and `.gitattributes`. Before building, **open each config file and read it**. You will be asked about them in the viva.

File	What it does
`config/Dockerfile`	Builds one image: Ubuntu 22.04, Java 11, Hadoop 3.3.6, Spark 3.5.1, Hive 3.1.3, Python packages (pandas, pymongo, pyarrow). Copies the configs and scripts last.
`config/docker-compose.yml`	Defines the 4 containers, memory caps, published ports, named volumes (data survives restarts) and the network.
`config/hadoop/core-site.xml`	`fs.defaultFS = hdfs://master:9000`: where HDFS lives.
`config/hadoop/hdfs-site.xml`	Replication 2, block size 16 MB, data folders, permissions off.
`config/hadoop/yarn-site.xml`	ResourceManager host, memory and CPU each worker offers, memory checks off.
`config/hadoop/mapred-site.xml`	MapReduce runs on YARN (used by Hive queries); small containers.
`config/hadoop/capacity-scheduler.xml`	Single queue; raises the share allowed for application masters.
`config/hadoop/hadoop-env.sh`	Java path, run-as-root flags, small JVM heaps.
`config/spark/spark-defaults.conf`	YARN client mode, 2 executors of 1 GB, driver host `master`, jars in HDFS.
`config/hive/hive-site.xml`	Derby metastore, warehouse folder, HiveServer2 on port 10000.

File	What it does
`scripts/entrypoint.sh`	At container start: formats the NameNode once, starts the right daemons.
`scripts/init_cluster.sh`	Waits for 2 DataNodes, creates HDFS folders, uploads Spark jars.
`scripts/start_hive.sh`	Initialises the metastore once and starts HiveServer2 in the background.
`scripts/cluster_status.sh`	Prints the evidence output (report, YARN nodes, block layout).

Commit the files in separate, meaningful commits (this helps your commit count and history):

```
git add config\Dockerfile config\docker-compose.yml .dockerignore .gitattributes
git commit -m "A: add Dockerfile and compose for 3-node cluster plus MongoDB"
git add config\hadoop
git commit -m "A: add Hadoop configs (HDFS replication 2, YARN resources)"
git add config\spark config\hive
git commit -m "A: add Spark-on-YARN and Hive configs"
git add scripts docs\cluster-setup.md
git commit -m "A: add cluster init, start and status scripts"
git push
```

A3. Build the image (11:00 to 11:30, runs in the background)

```
cd C:\Users\Lenovo\BigDataProject\config
docker compose build
```

- First build takes **15 to 25 minutes** and downloads about 1.5 GB (Hadoop, Spark, Hive). Do not close the window.
- While it builds, do A4 preparation: get `quakes.csv` from Person B (link or USB) and place it in `C:\Users\Lenovo\BigDataProject\dataset\quakes.csv`. It is ignored by Git.
- Check the size: it should be roughly 80 to 150 MB for 500,000+ rows.

Note: If a download fails (slow `archive.apache.org`), just run the build again; finished steps are cached. If it keeps failing, download the three files in a browser (hadoop-3.3.6.tar.gz, spark-3.5.1-bin-hadoop3.tgz, apache-hive-3.1.3-bin.tar.gz) into `config/downloads/` and ask me to switch the Dockerfile from `curl` to `COPY`.

A4. Start the cluster and initialise HDFS (11:30 to 13:00)

```
docker compose up -d
docker compose ps                       # 4 containers: master, worker1, worker2, mongo
docker logs master                      # should end with "NameNode + ResourceManager started"
docker logs worker1                     # "DataNode + NodeManager started"
```

Wait about one minute, then initialise HDFS (once):

```
docker exec master bash /opt/project/scripts/init_cluster.sh
```

Expected: it waits for 2 live DataNodes, creates `/earthquake/...`, uploads the Spark jars (1 to 3 minutes) and prints `INIT DONE`. Now verify:

```
docker exec master jps          # NameNode, ResourceManager
docker exec worker1 jps         # DataNode, NodeManager
docker exec worker2 jps         # DataNode, NodeManager
docker exec master hdfs dfsadmin -report | findstr /C:"Live datanodes" /C:"Name:"
docker exec master yarn node -list
```

Expected: `Live datanodes (2)` and two `RUNNING` nodes. Open `http://localhost:9870` (Datanodes tab) and `http://localhost:8088` (Nodes). **Take your first screenshots now** (section 5).

Optional but recommended: add this line to `C:\Windows\System32\drivers\etc\hosts` (open Notepad as administrator) so that links inside the web UIs work in your browser:

```
127.0.0.1  master worker1 worker2
```

Commit: `A: add cluster status screenshots (jps, report, YARN nodes)` after copying PNGs to `screenshots\`.

A5. Run a Spark job on YARN (14:00 to 14:45)

```
docker exec master spark-submit --class org.apache.spark.examples.SparkPi ^
  /opt/spark/examples/jars/spark-examples_2.12-3.5.1.jar 100
```

- Expected last lines: `Pi is roughly 3.14...`. The first run takes about 30 to 60 seconds.
- While it runs, open `http://localhost:8088` and watch the application go from ACCEPTED to RUNNING.
- Open `http://localhost:4040` (Spark UI) during the run, tab **Executors**: you should see executors on `worker1` and `worker2`. Screenshot it.
- If it hangs at ACCEPTED, see Troubleshooting (section 11).

Also test the Python path, because B and C use PySpark:

```
docker exec master bash -c "echo 'from pyspark.sql import SparkSession;
s=SparkSession.builder.getOrCreate(); print(s.range(1000000).count())' > /tmp/t.py && spark-submit
/tmp/t.py"
```

Expected: `1000000` in the output. Tell B and C: **"Spark on YARN works, you can run jobs."**

A6. Upload the full dataset to HDFS (14:45 to 15:45)

Create `src/ingestion/load_to_hdfs.sh`:

```
#!/usr/bin/env bash
# Usage (inside master): bash /opt/project/src/ingestion/load_to_hdfs.sh [path-to-csv]
set -euo pipefail
SRC=${1:-/opt/project/dataset/quakes.csv}
DEST=/earthquake/raw

hdfs dfs -mkdir -p "$DEST"
hdfs dfs -D dfs.blocksize=16777216 -D dfs.replication=2 -put -f "$SRC" "$DEST/quakes.csv"

echo "---- listing ----";      hdfs dfs -ls -h "$DEST"
echo "---- first 3 lines ----"; hdfs dfs -cat "$DEST/quakes.csv" | head -3
echo "---- block layout ----";  hdfs fsck "$DEST/quakes.csv" -files -blocks -locations
```

```
docker exec master bash /opt/project/src/ingestion/load_to_hdfs.sh
```

What to look for in the `fsck` output:

- Several lines like `blk_...: len=16777216 Live_repl=2` (a 100 MB file gives about 7 blocks).
- Each block lists two locations, one on `worker1` and one on `worker2` (addresses ending in `:9866`).
- Summary: `Average block replication: 2.0`, `Under-replicated blocks: 0`, `Number of data-nodes: 2`.

Open `http://localhost:9870`, Utilities, Browse the file system, `/earthquake/raw/quakes.csv`: the page lists each block and the DataNodes holding it. Screenshot both views. **Commit:** `A: add HDFS ingestion script and block distribution evidence`

> **Note:** Tell Person B right away: `quakes.csv` is in HDFS, you can run `clean_data.py` on the cluster.

A7. Hive (15:45 to 16:45)

```
docker exec master bash /opt/project/scripts/start_hive.sh
```

First start initialises the metastore and takes 1 to 2 minutes. Expected last line: `HiveServer2 is up on port 10000`. Test with a throw-away table:

```
docker exec master beeline -u jdbc:hive2://master:10000 -e "SHOW DATABASES;"
docker exec master beeline -u jdbc:hive2://master:10000 -e "CREATE DATABASE IF NOT EXISTS test_db; SHOW DATABASES;"
docker exec master beeline -u jdbc:hive2://master:10000 -e "DROP DATABASE test_db;"
```

Expected: `default` (and `test_db` while it exists). The `beeline` command in the image already adds `-n root`, so no username prompt appears. B's DDL will be tested on Day 2 (A11).

If HiveServer2 does not start, read `docker exec master tail -50 /var/log/hiveserver2.log`.

A8. MongoDB check (16:45 to 17:15)

```
docker exec mongo mongosh --quiet --eval "db.runCommand({ping:1})"
docker exec master python3 -c "from pymongo import MongoClient; c=MongoClient('mongodb://mongo:27017'); c.test.t.insert_one({'ok':1}); print(c.test.t.count_documents({})); c.drop_database('test')"
```

Expected: `{ ok: 1 }`, then `1`. This proves master can reach `mongo` by name, which is what C's loader needs (`MONGO_URI=mongodb://mongo:27017`). From Windows (for Compass) use `localhost:27017`.

A9. Evidence, docs and the 18:00 sync (17:15 to 18:00)

- Run `docker exec master bash /opt/project/scripts/cluster_status.sh` and screenshot the full output.
- Update `docs/cluster-setup.md` with the real versions and anything you changed.
- Check memory: `docker stats --no-stream` (total should stay under 8.5 GB while idle, usually about 3 GB).
- Move all screenshots into `screenshots\` with the names in section 5, then commit.
- Tell B and C how they reach the cluster (section 6) and which command opens a shell: `docker exec -it master bash`.
- **18:00 sync (15 min):** confirm paths (`/earthquake/raw`, `/earthquake/clean`, `/earthquake/output`, `/earthquake/ml`), the repo mount `/opt/project`, and `MONGO_URI=mongodb://mongo:27017`.

Commit: `A: add cluster-setup docs and Day 1 evidence`

Day 1 end-of-day checklist

[ ] Repo exists, B and C are collaborators and have cloned

[ ] 4 containers running; `jps` correct on all 3 nodes

[ ] `Live datanodes (2)`, 2 YARN nodes RUNNING

[ ] SparkPi ran on YARN with executors on both workers

[ ] `quakes.csv` in HDFS; `fsck` shows blocks on both workers

[ ] HiveServer2 up; MongoDB reachable from master

[ ] At least 6 commits under your account

4. Day 2 tasks

Goal for the end of Day 2: B's and C's jobs ran on your cluster, Hive and MongoDB hold results, the failure test is documented, all evidence is in the repo, and the full pipeline runs with one command.

A10. Morning health check (09:00 to 09:30)

```
docker compose ps
docker exec master hdfs dfsadmin -report | findstr /C:"Live datanodes"
docker exec master hdfs dfs -ls /earthquake/raw
docker stats --no-stream
```

- If you restarted the PC overnight: `cd config`, `docker compose up -d`, wait 1 minute, then run `start_hive.sh` again (Hive does not start automatically). HDFS may stay in safe mode for up to a minute: `docker exec master hdfs dfsadmin -safemode wait`.
- Pull teammates' work: `git pull --rebase` on Windows.
- Check commit counts: `git shortlog -sn` (everyone should have at least 4 by now).

A11. Support B and C on the cluster (09:30 to 13:00)

Your most important job on Day 2. B runs the cleaning (about 09:00), then analytics and Hive; C runs `select_k.py` and the clustering. Rules that keep the cluster healthy:

1. **One job at a time.** Agree a simple order: B cleaning, then B analytics, then C k-selection, then C clustering. Hive queries can overlap with nothing heavy.
2. Watch the YARN UI (`localhost:8088`): a job in ACCEPTED for more than 2 minutes means no free memory.
3. Watch `docker stats` in another window: if a worker approaches its 2.7 GB cap, wait for the job to end.
4. After B finishes `clean_data.py`, run B's Hive file once together and verify counts:

```
docker exec master beeline -u jdbc:hive2://master:10000 -f /opt/project/src/analytics/hive_queries.hql
docker exec master beeline -u jdbc:hive2://master:10000 -e "SELECT COUNT(*) FROM quakes_db.quakes;"
```

The Hive count must equal `CLEAN ROWS` printed by Spark. If Hive returns 0, run `MSCK REPAIR TABLE quakes_db.quakes;`. Hive aggregation queries run as MapReduce jobs on YARN (each takes 20 to 60 seconds), which is also evidence of distributed processing. Screenshot the YARN UI during a Hive query.

Commit: `A: add cluster tuning notes and Hive verification screenshots`

A12. Failure test: lose a worker (14:00 to 15:00)

This shows that HDFS and YARN tolerate a node failure, which is a strong point in the viva. With only 2 DataNodes and replication 2, the lost blocks cannot be copied elsewhere, so the data becomes *under-replicated* but stays readable. When the worker returns, it heals automatically.

Step 1: make failure detection fast (config change)

By default HDFS waits about 10 minutes before declaring a DataNode dead, and YARN waits 10 minutes for a NodeManager. Shorten both for the demo. Add inside `<configuration>` in `config/hadoop/hdfs-site.xml`:

```
<property><name>dfs.namenode.heartbeat.recheck-interval</name><value>10000</value></property>
```

and in `config/hadoop/yarn-site.xml`:

```
<property><name>yarn.nm.liveness-monitor.expiry-interval-ms</name><value>30000</value></property>
```

Rebuild (seconds, only the last layers change) and restart. Data in volumes is kept:

```
cd C:\Users\Lenovo\BigDataProject\config
docker compose build
docker compose up -d
docker exec master bash /opt/project/scripts/start_hive.sh
```

Note: The shorter timeouts make a node look dead after about 50 seconds without heartbeats. Under heavy load that can cause false alarms, so use these settings only for the failure test, then revert the two lines (and rebuild) before the final pipeline run, or leave them if all runs stay stable.

Step 2: run the test

```
docker stop worker2
:: wait about 60 seconds
docker exec master hdfs dfsadmin -report | findstr /C:"Live datanodes" /C:"Dead datanodes"
docker exec master yarn node -list -all
docker exec master bash -c "hdfs dfs -cat /earthquake/raw/quakes.csv | head -2"
docker exec master hdfs fsck /earthquake/raw -files -blocks | findstr /i "under-replicated"
```

- Expected: `Live datanodes (1)`, `Dead datanodes (1)`; YARN lists worker2 as LOST (or UNHEALTHY).
- The `cat` still prints data: the surviving replica serves every block.
- `fsck` reports under-replicated blocks (each block has 1 of 2 replicas).
- Run a job on the surviving node: `SparkPi` with 20 tasks. It still finishes, with one executor.

```
docker exec master spark-submit --class org.apache.spark.examples.SparkPi ^
  /opt/spark/examples/jars/spark-examples_2.12-3.5.1.jar 20
```

Step 3: bring the worker back

```
docker start worker2
:: wait about 60 seconds
docker exec master hdfs dfsadmin -report | findstr /C:"Live datanodes"
docker exec master hdfs fsck /earthquake/raw | findstr /i "replicat"
```

Expected: `Live datanodes (2)` and `Under-replicated blocks: 0` again. Screenshot each of the three stages, and write a 10-line `docs/failover-test.md` (what we did, what we saw, what it proves). **Commit:** `A: add failover test (worker loss) with evidence`

A13. Final evidence set (15:00 to 16:00)

Re-take any screenshot that looks unclear, using the exact list in section 5. Use a clean window at 100% zoom, with the command and the output both visible, and the date and time on screen if possible. Take the Spark UI screenshot **during** a real project job (for example B's `analytics_full.py`), not only SparkPi.

Commit: `A: add final cluster evidence screenshots`

A14. Full pipeline dry run (16:00 to 16:45)

With B and C present, run C's single command and time it:

```
docker exec master bash /opt/project/pipeline/run_pipeline.sh
```

- Your jobs: watch `docker stats` and the YARN UI; do not touch the cluster while it runs.
- If a step fails, copy the last 30 lines of the output and the failing script name; fix only blockers.
- Write down the total time. If it is above 10 minutes, plan the demo to show a **recorded run** (screenshots of each step) plus one live step instead of the whole pipeline.
- Before the demo: the cluster should be freshly restarted, jars already in HDFS, Hive already started.

A15. Demo script and rehearsal (16:45 to 18:00)

Write `docs/demo-script.md` from section 9 and rehearse twice with a timer. Your part: about 3 minutes. Also run the **final repo check** (section 12) and fix gaps. **Commit:** `A: add demo script for cluster part`

Day 2 end-of-day checklist

[ ] B's and C's jobs ran with `--master yarn`; Spark UI screenshot shows executors on both workers

[ ] Hive count equals Spark count; Hive query ran on YARN

[ ] Failover test documented with 3 screenshots

[ ] All 5 course items have screenshots in `screenshots/`

[ ] Pipeline ran end to end at least once

[ ] Every member has at least 5 commits (you have 10 or more)

[ ] Repo check (section 12) is all ticked

5. Screenshot checklist (the 5 things the course asks to see)

Save as PNG in `screenshots\` with these names (Windows key + Shift + S). In the README, C will embed them.

Course item	File name	Command or page	Must be visible
1. Node configuration	`01-compose-nodes.png`	Open `config/docker-compose.yml`	master, worker1, worker2 services with roles
1. Node configuration	`02-hdfs-config.png`	`docker exec master hdfs getconf -confKey dfs.replication` and `dfs.blocksize`	Replication 2, block size 16777216
2. Cluster status	`03-jps-master.png`, `03-jps-worker1.png`, `03-jps-worker2.png`	`docker exec <node> jps`	NameNode and ResourceManager; DataNode and NodeManager
2. Cluster status	`04-dfsadmin-report.png`	`hdfs dfsadmin -report`	Live datanodes (2) with both host names
2. Cluster status	`05-yarn-nodes.png`	`yarn node -list` and `localhost:8088/cluster/nodes`	2 RUNNING nodes
2. Cluster status	`06-namenode-ui.png`	`localhost:9870`, Datanodes tab	Two DataNodes in service
3. Storage distribution	`07-fsck-blocks.png`	`hdfs fsck /earthquake/raw/quakes.csv -files -blocks -locations`	Blocks with two locations; replication 2
3. Storage distribution	`08-hdfs-browse-blocks.png`	`localhost:9870`, Utilities, Browse, file page	Block list with hosts
4. Analytics execution	`09-spark-submit.png`	`spark-submit ... clean_data.py` (B's job)	`--master yarn` command and the finish line
4. Analytics execution	`10-spark-ui-executors.png`	`localhost:4040`, Executors tab (during the job)	Executors on worker1 and worker2
4. Analytics execution	`11-yarn-apps.png`	`localhost:8088`	Finished application list, state SUCCEEDED
4. Analytics execution	`12-hive-beeline.png`	A Hive query in Beeline	Query and result rows
5. Output and results	`13-top-regions.png`	`head output/top_regions.csv`	Hotspot regions with counts
5. Output and results	`14-cluster-map.png`	C's `cluster_map.html` in a browser	Cluster map
5. Output and results	`15-mongo-queries.png`	`python3 src/analytics/mongo_queries.py`	Query output
Bonus	`16-failover-*.png`	Failure test steps 2 and 3	Dead node, readable data, recovery

6. Serving B and C (hand-off and remote access)

B and C write scripts on their laptops (Spark local mode on the 10,000-row sample) and need to run them on your cluster. You choose one of these ways.

Option 1: they sit with you (simplest and most reliable)

```
docker exec -it master bash
cd /opt/project
# now they type or paste their spark-submit commands
```

Option 2: screen share

Share your screen on Teams, Zoom or Meet and let them dictate commands. Works with no setup.

Option 3: remote shell with Tailscale (set up on Day 1 evening, optional)

1 Install **Tailscale** on your PC and on B's and C's laptops; sign in with the same account (or share the device with them).

2 On your PC, install OpenSSH Server: Settings, Apps, Optional features, Add a feature, **OpenSSH Server**. Then in an administrator PowerShell: `Start-Service sshd` and `Set-Service -Name sshd -StartupType Automatic`.

3 They connect: `ssh Lenovo@<your-tailscale-ip>`, then `docker exec -it master bash`.

4 Test it once yourself from another device before relying on it.

Note: Keep your PC awake and plugged in. Give them access only for the project days and remove them from Tailscale afterwards.

The loop everyone follows

1 They commit and push a script from their laptop.

2 You run `git pull --rebase` on Windows. The folder is live inside `/opt/project`.

3 They run it with `spark-submit` in the master container (one job at a time).

4 They check results and fix scripts on their laptop.

7. Daily operation cheat sheet

What you want	Command (run in `config\` unless noted)
Start everything	`docker compose up -d`, wait 1 minute, then `docker exec master bash /opt/project/scripts/start_hive.sh`
Stop everything (keep data)	`docker compose stop`
Start again after stop	`docker compose start`, then start Hive again
Open a shell in master	`docker exec -it master bash`
See running containers	`docker compose ps`
Memory use	`docker stats --no-stream`
Container logs	`docker logs master` (also `worker1`, `worker2`, `mongo`)
HiveServer2 log	`docker exec master tail -50 /var/log/hiveserver2.log`
List HDFS files	`docker exec master hdfs dfs -ls -R /earthquake`
Free HDFS space	`docker exec master hdfs dfs -rm -r -skipTrash /earthquake/ml`
Kill a stuck Spark job	`docker exec master yarn application -list` then `yarn application -kill <id>`
Rebuild after config change	`docker compose build` then `docker compose up -d`
Wipe everything (DANGER)	`docker compose down -v` deletes HDFS, Hive and Mongo data

8. Commit plan

Commit from Windows after each task. Short, specific messages.

#	When	Commit message	Files
1	D1 09:45	A: create repo skeleton and gitignore	.gitignore, .gitkeep files
2	D1 11:00	A: add Dockerfile and compose for 3-node cluster plus MongoDB	config/Dockerfile, docker-compose.yml
3	D1 11:00	A: add Hadoop configs (HDFS replication 2, YARN resources)	config/hadoop/
4	D1 11:00	A: add Spark-on-YARN and Hive configs	config/spark/, config/hive/
5	D1 11:00	A: add cluster init, start and status scripts	scripts/
6	D1 13:00	A: add cluster status screenshots	screenshots/03 to 06
7	D1 15:45	A: add HDFS ingestion script and block distribution evidence	src/ingestion/, screenshots/07, 08
8	D1 18:00	A: add cluster-setup docs and Day 1 evidence	docs/cluster-setup.md
9	D2 13:00	A: add Hive verification and Spark UI screenshots	screenshots/09 to 12
10	D2 15:00	A: add failover test (worker loss) with evidence	docs/failover-test.md, screenshots/16
11	D2 16:00	A: add final cluster evidence screenshots	screenshots/
12	D2 18:00	A: add demo script for cluster part	docs/demo-script.md

9. Demo narration (about 3 minutes)

You open the demo, so the first impression is yours. Do every action on a clean terminal.

Step	Say	Show
1	"Our cluster has one master and two worker nodes, running as Docker containers on this PC."	`docker compose ps` and the compose file
2	"Node configuration: HDFS replication is 2 and the block size is 16 MB because we have two DataNodes."	`hdfs getconf -confKey dfs.replication` and `dfs.blocksize`
3	"The master runs NameNode and ResourceManager, each worker runs a DataNode and a NodeManager."	`docker exec master jps`, `worker1 jps`, `worker2 jps`
4	"The cluster reports two live DataNodes and two running YARN nodes."	`hdfs dfsadmin -report`, `yarn node -list`
5	"The dataset of N earthquake records is split into blocks and each block is stored on both workers."	`hdfs fsck /earthquake/raw/quakes.csv -files -blocks -locations`
6	"Spark runs on YARN: the driver is on the master and executors run on both workers."	Start a `spark-submit` and open `localhost:4040`, Executors tab
7	"Even if a worker fails, data stays readable and jobs continue." (if time)	Show the failover screenshots, or run `docker stop worker2` and `hdfs dfs -cat`
8	"Now Person B will show the processing."	Hand over

10. Viva questions you should be able to answer

Q: What does the NameNode do, and what do DataNodes do?

A: The NameNode stores the metadata (which blocks make up each file and where they are). DataNodes store the actual blocks and send heartbeats and block reports to the NameNode.

Q: Why replication factor 2?

A: We have only two DataNodes. The default of 3 could not be satisfied, so blocks would stay under-replicated. With 2, every block lives on both workers, which is also easy to show with `fsck`.

Q: Why a 16 MB block size?

A: The default is 128 MB, so our 100 MB file would be one block on one pair of nodes. A smaller block makes the file split into several blocks, showing the storage distribution. Block size also sets the number of Spark input partitions.

Q: What does YARN do?

A: YARN is the cluster resource manager. The ResourceManager decides which NodeManager runs which container; NodeManagers run the containers (Spark executors, MapReduce tasks).

Q: What is the difference between client and cluster deploy mode?

A: In client mode the Spark driver runs where you submit (here the master container); in cluster mode the driver runs inside YARN. Client mode is easier to debug and lets scripts import local modules.

Q: What is data locality?

A: Spark and YARN try to run each task on a node that already stores the block, so data does not travel over the network.

Q: What happens if the NameNode fails?

A: In this setup HDFS stops, because the NameNode is a single point of failure. Production clusters use NameNode high availability with a standby. Data is safe on disk (the metadata is in a Docker volume).

Q: What happens if one DataNode fails?

A: The NameNode notices missing heartbeats, marks the node dead and serves reads from the other replica. With a third node it would re-copy the blocks; with two, blocks stay under-replicated until the node returns.

Q: What is safe mode?

A: At start the NameNode is read-only until enough DataNodes have reported their blocks. We wait for it with `hdfs dfsadmin -safemode wait`.

Q: Why did you put Spark's jars in HDFS?

A: Without it every job uploads about 300 MB of jars to YARN. With `spark.yarn.jars` pointing to HDFS the jars are uploaded once, so jobs start in seconds.

Q: Why Docker containers and not virtual machines?

A: Containers share the host kernel, start in seconds and need less memory. The course allows VMs, Docker containers or cloud instances as long as there are 1 master and 2 workers with distributed execution.

Q: Why is Hive an external table and where is its metadata?

A: External because Spark wrote the data and we do not want Hive to delete it. The metadata (schema, partitions) is in the Derby metastore in a Docker volume; the data stays in HDFS.

11. Troubleshooting

Problem	Fix
`docker compose build` fails while downloading	Run it again (cached steps are skipped). If it keeps failing, download the tarballs manually (see note in A3).
Container `master` exits immediately	`docker logs master`. A `\r` or `bad interpreter` error means Windows line endings: run `git add --renormalize .` or re-unzip, then rebuild.
Container exits with code 137	Out of memory. Close other apps, check `docker stats`, make sure `.wslconfig` has 9 GB, run `wsl --shutdown`, restart Docker.
`Live datanodes (0)` or only 1	`docker logs worker1`. Wait one minute after `up`. If it says "Incompatible clusterIDs", you re-formatted only the NameNode: wipe with `docker compose down -v` and start fresh (data is lost).
NameNode stuck in safe mode	`docker exec master hdfs dfsadmin -safemode wait` (or `leave` if blocks are fine).
Spark job stays in ACCEPTED	Check `yarn node -list` (2 RUNNING?) and that no other job is running. Kill stale apps with `yarn application -kill`. Lower memory flags only if you changed defaults.
Spark error: `File /spark-jars/... does not exist`	`init_cluster.sh` did not finish. Run it again.
Spark: `Connection refused ... master:<port>` from executors	The driver was not reachable. Check `spark.driver.host master` is in `spark-defaults.conf` and that the job is submitted from inside the master container.
Beeline: `Connection refused` on 10000	HiveServer2 not running. `start_hive.sh`; read `/var/log/hiveserver2.log`.
Hive: guava or NoSuchMethodError at start	The guava jar swap in the Dockerfile did not apply; rebuild with `docker compose build --no-cache` (long) or tell me the log lines.
Hive query returns 0 rows	Run `MSCK REPAIR TABLE quakes_db.quakes;` and check the `LOCATION`.
Hive query very slow	Normal: each query is a MapReduce job (20 to 60 s). Do not run it during Spark jobs.
Web UI links open `master:8088` and fail	Add the hosts-file line from A4.
Port already in use (9870, 8088, 27017...)	`netstat -ano | findstr :9870`, stop that program or change the left side of the port mapping in the compose file.
PC freezes during jobs	Reduce load: only one job at a time; set `.wslconfig` memory to 8GB; close apps.
`git pull` shows conflicts in `output/`	Take their version of generated CSV files (`git checkout --theirs`) and commit.

12. Final repo check (Day 2, 17:30)

You own repo health. Run these on Windows from the repo folder:

```
git pull --rebase
git shortlog -sn --all                      :: commits per person (each must be at least 5)
git log --format="%an %ad %s" --date=short  :: commits spread across both days
dir /s /b dataset                           :: only sample.csv should be tracked
git ls-files | findstr /v ".gitkeep"
```

[ ] Each member has at least 5 meaningful commits from their own account (no single final dump)

[ ] Folder structure matches the course brief: `dataset/ src/{ingestion,processing,analytics}/ scripts/ config/ output/ screenshots/ docs/ pipeline/`

[ ] `README.md` renders on GitHub with architecture, cluster specs, run steps and screenshots

[ ] `screenshots/` has all 5 required items (section 5)

[ ] No file larger than 100 MB is tracked (`dataset/quakes.csv` is ignored)

[ ] `pipeline/run_pipeline.sh` is committed and has run once successfully

[ ] Roles table in the README matches who actually did what

[ ] Repo is private and all three members can open it from their own accounts

Note: Course rule: contribution is judged by GitHub commit history. Make sure B and C also see these reminders: use their own accounts, commit throughout, never push everything in one commit.